

Refatoração com Adapter e Decorator

Relatório Técnico - SI405

Disciplina: SI405 - Análise de Sistemas de Informação II

Atividade: Padrões Estruturais - Adapter e Decorator

Data: 25/06/2026

Integrantes:

Nome	RA
Daniel Aniceto Rosell	283988
Davie Schimidt Fonseca	259908
Hugo Strassa	246710
Gabriel Sorensen M Traina	283997
Kaue Samuel Oliveira da Silva	178449
Kauã Henrique da Silva Andrade	246165

Limeira - SP
2026

Repositório GitHub da solução: <https://github.com/SorensenG/si405-padroes-estruturais-grupo>

O desenvolvimento foi organizado entre 18/06/2026 às 14:00 e 25/06/2026, com commits separados por etapa e coautorias registradas conforme a participação dos integrantes.

1. Análise dos principais problemas arquiteturais identificados

O sistema legado funcionava corretamente, mas o serviço de cobrança concentrava responsabilidades que deveriam estar separadas. A classe `CobrancaService` conhecia diretamente a implementação do gateway interno, a biblioteca externa `PaySecureGateway`, o formato de dados esperado por ela e o tratamento de exceções específico do SDK. Com isso, uma regra de aplicação ficava acoplada a detalhes de integração de infraestrutura.

Outro problema estava na duplicação da lógica de seleção e conversão de gateways. Os métodos `cobrar` e `cobrarEmLote` repetiam a mesma estrutura condicional para Boleto, Pix e Cartão de Crédito. Qualquer nova forma de pagamento exigiria alterar os dois métodos, aumentando o risco de inconsistências e dificultando a evolução do sistema.

No cálculo do valor final, a combinação de ajustes era representada por parâmetros booleanos. Essa abordagem deixa a assinatura do método crescer conforme novos ajustes surgem e mistura a regra de composição com a regra de cálculo. A adição de taxa de antecipação de recebíveis e taxa de emissão de nota fiscal tornaria o problema ainda mais evidente se a solução continuasse baseada em novos booleanos.

2. Justificativa das decisões arquiteturais adotadas

A solução manteve o projeto original e realizou uma refatoração localizada, preservando o comportamento funcional existente e os testes legados. Em vez de substituir o sistema por outro, a implementação introduziu abstrações nos pontos em que havia maior acoplamento: integração com gateways de pagamento e composição de ajustes de valor.

Para os meios de pagamento, foi adotado o padrão `Adapter`. O `CobrancaService` passou a depender de uma interface comum, `GatewayCobranca`, enquanto cada integração concreta ficou encapsulada em um adapter. Essa escolha permite que bibliotecas externas com interfaces incompatíveis sejam usadas sem modificar seu código, respeitando a restrição do enunciado.

Para os ajustes de valor, foi adotado o padrão `Decorator`. O valor base passou a ser representado por um componente simples, e cada ajuste passou a envolver esse componente adicionando sua regra de cálculo. Com isso, os ajustes podem ser combinados em qualquer quantidade e ordem, sem alterar a assinatura principal de cobrança quando um novo ajuste é criado.

3. Explicação da aplicação do padrão Adapter

A interface `GatewayCobranca` define uma operação comum de cobrança recebendo `Pedido`, valor final e `FormaPagamento`. O `GatewayPagamentoInternoAdapter` adapta o gateway interno usado por Boleto e Pix; o `PaySecureGatewayAdapter` adapta o SDK externo de cartão de crédito; e o `WalletPayGatewayAdapter` integra a nova Carteira Digital solicitada na atividade.

No adapter do `PaySecureGateway`, a solução centraliza a montagem do Map exigido pelo SDK, com campos como `orderId`, `customerName`, `amount` e `currency`. Também fica nesse adapter a conversão do código de status externo para os status internos `APROVADA` e `RECUSADA`, além do tratamento de `GatewayIndisponivelException`.

No adapter do WalletPaySDK, a solução converte o valor em reais para centavos, cria o ChargeRequest esperado pela biblioteca externa e traduz ChargeStatus.CONFIRMED para APROVADA. Status DECLINED ou FAILED são tratados como RECUSADA. O CobrancaService não conhece esses detalhes e apenas solicita ao GatewayCobrancaResolver o gateway compatível com a forma de pagamento escolhida.

4. Explicação da aplicação do padrão Decorator

O cálculo do valor final foi reorganizado a partir da interface ValorCobranca. A classe ValorBaseCobranca representa o valor inicial do pedido, e AjusteValorDecorator serve como classe base para todos os decorators. Cada decorator recebe um ValorCobranca e acrescenta sua regra ao resultado já calculado pelo objeto decorado.

Foram criados decorators para os ajustes já existentes: DescontoFidelidadeDecorator, JurosParcelamentoDecorator, TaxaOperacaoInternacionalDecorator e SeguroTransacaoDecorator. As regras e percentuais originais foram preservados: desconto de 5%, juros de 2,99%, taxa internacional de 5% e seguro fixo de R\$ 4,90.

Também foram adicionados os novos ajustes exigidos: TaxaAntecipacaoRecebeveisDecorator, com acréscimo de 1,5% sobre o valor, e TaxaEmissaoNotaFiscalDecorator, com valor fixo de R\$ 2,50. O CobrancaService agora aceita uma lista de AjusteValor, permitindo montar a cadeia de decorators dinamicamente. Os métodos antigos com booleanos foram mantidos apenas como compatibilidade e delegam para a nova composição.

5. Impactos da refatoração sobre organização, manutenção e extensibilidade

A refatoração reduziu o acoplamento do serviço de cobrança com gateways concretos. Para adicionar uma nova forma de pagamento, o fluxo esperado passa a ser criar um novo adapter, registrar a forma no resolver e implementar testes específicos. A regra principal de cobrança permanece estável.

A duplicação entre cobrança única e cobrança em lote também foi reduzida. Os dois fluxos reutilizam a mesma lógica de cálculo e a mesma resolução de gateway, evitando que mudanças futuras precisem ser repetidas manualmente em mais de um método.

Com Decorator, os ajustes de valor ficaram independentes e combináveis. A criação de novas taxas não exige novos parâmetros booleanos nem condicionais no cálculo central. Essa organização torna o sistema mais extensível, facilita testes isolados e deixa mais clara a intenção arquitetural discutida em aula: preferir composição sobre herança ou condicionais rígidos.

Testes executados

A suíte automatizada foi ampliada para contemplar o comportamento dos adapters do PaySecureGateway e do WalletPaySDK, os decorators novos e a composição de múltiplos decorators em diferentes ordens. A execução local de mvn test finalizou com 34 testes, 0 falhas e 0 erros.

Divisão de responsabilidades

Parte	Responsáveis
Estrutura base de adapters e gateway interno	Daniel, Gabriel
Adapter do PaySecureGateway	Davie, Gabriel
Integração do WalletPaySDK e Carteira Digital	Hugo, Gabriel
Refatoração do CobrancaService e resolver de gateways	Daniel, Davie, Hugo, Gabriel
Decorators dos ajustes existentes	Kaue, Gabriel
Decorators novos, testes e revisão final	Kauã, Kaue, Gabriel e demais integrantes